CAPSTONE PROJECT 3

Agentic ETL Pipeline with PySpark, Databricks, ADLS and Microsoft Foundry

Project: uber-taxi-agentic-etl

Uber Delhi NCR ride bookings · Medallion architecture · LangGraph multi-agent system

Submitted by	Rahul
Department	Information Technology
Institution	Chandigarh Engineering College-CGC, Landran, Mohali, Punjab
Report date	14 October 2026
Cloud platform	Microsoft Azure (storage account azstore01, workspace myazdbworkspace)
Technology stack	PySpark · Azure Databricks · ADLS Gen2 · Delta Lake · LangGraph · Microsoft Foundry (gpt-5-mini)

Contents

1. Problem Statement and Objectives
2. Architecture
3. Dataset and ADLS Layout
4. Data Transformation Logic (Bronze, Silver, Gold)
5. Data Quality Framework
6. Databricks Environment and Test Results
7. Agent Design: Roles, Tools, Workflow, Guardrails
8. Foundry GPT Deployment and Prompt Design
9. Fault Injection and Evaluation
10. Challenges, Limitations and Future Work
11. Conclusion
12. Appendix: Source Code Map

Reading guide: This report documents the pipeline exactly as implemented in the submitted source code and as shown in the screenshots. Where the implementation is narrower than the project guideline, the gap is stated openly in the relevant section and consolidated in Section 10.

1. Problem Statement and Objectives

Data pipelines can finish with status “Succeeded” while still delivering duplicate rows, missing days, impossible fares or shrunken record counts. Engineers then spend hours on manual checks, log reading and re-runs. This project addresses that gap for Uber taxi ride bookings in Delhi NCR (cars, autos and bikes) in two phases.

1.1 Objectives

- **Phase 1 – ETL pipeline.** Ingest the raw booking CSV from Azure Data Lake Storage Gen2, transform it with PySpark on Azure Databricks through Bronze, Silver and Gold Delta layers, and validate the output.
- **Phase 2 – Multi-agent layer.** Build a LangGraph team of agents (supervisor, etl, quality, analytics, explain), powered by a GPT model on Microsoft Foundry, that runs the pipeline, validates it and answers questions in plain language.

1.2 Scope delivered

Area	Delivered in this submission
Storage	ADLS Gen2 account azstore01 (hierarchical namespace enabled) with container uber-taxi holding the source CSV, a Power BI file and a dashboard GIF
Compute	Azure Databricks workspace myazdbworkspace; project folder uber-taxi-agentic-etl with data, logs, notebooks, src
ETL	Bronze, Silver and three Gold Delta tables written to ADLS via abfss paths (src/etl_pipeline.py)
Quality	Silver-layer validation function returning a structured pass/fail dictionary (src/quality_checks.py)
Agents	LangGraph StateGraph with supervisor, etl, quality, analytics and explain nodes on Foundry gpt-5-mini (src/agents.py)
Visuals	Power BI report (Uber.pbix) and dashboard GIF stored in the container

2. Architecture

The solution has two cooperating planes. The data plane stores and transforms data in ADLS Gen2 using Databricks and Delta Lake. The agent plane sits on top and calls the data-plane functions as tools, using the Foundry-hosted model for routing and for natural-language answers.

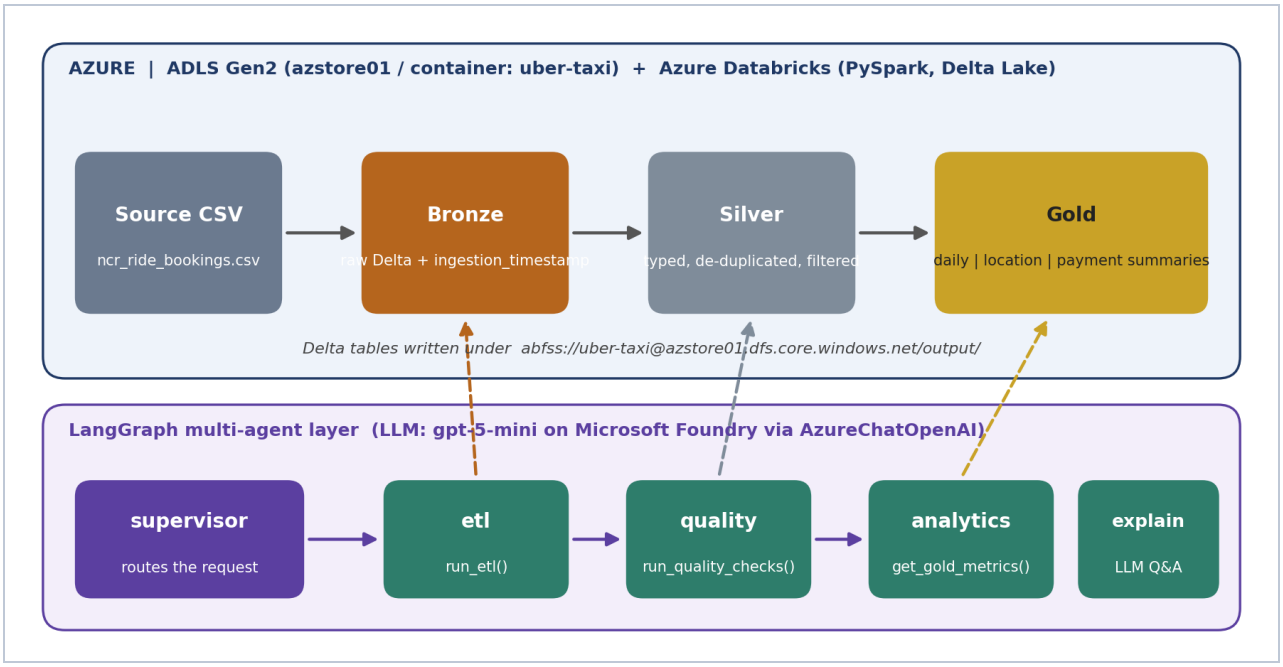


Figure 1: *End-to-end architecture: ADLS Gen2 and Databricks medallion layers (top) operated by the LangGraph agent layer (bottom). Diagram prepared for this report.*

2.1 Component summary

Component	Technology	Role
Storage	ADLS Gen2 (azstore01)	Holds source CSV and all Delta output under /output
Processing	Azure Databricks, PySpark 3.x, Delta Lake	Bronze → Silver → Gold transformations
Orchestration of agents	LangGraph 0.6.2, LangChain 0.3.27	Stateful graph with conditional routing
LLM	Microsoft Foundry, gpt-5-mini via AzureChatOpenAI	Route selection, analytics answers, explanations
BI	Power BI (Uber.pbix) and dashboard GIF	Business-facing presentation of Gold metrics

3. Dataset and ADLS Layout

The dataset contains Uber ride bookings for Delhi NCR, one row per booking, with 21 columns covering booking identity, status, vehicle type, locations, turnaround times (Avg VTAT, Avg CTAT), cancellation and incomplete-ride flags with reasons, booking value, ride distance, ratings and payment method. Outcome columns are populated only when the booking reached that outcome, so null handling is part of the business logic.

3.1 Azure storage account

The storage account azstore01 was created with a hierarchical namespace, which is what makes it an ADLS Gen2 account. Blob soft delete and container soft delete are enabled for 7 days, anonymous blob access is disabled, minimum TLS is 1.2, and secure transfer is required.

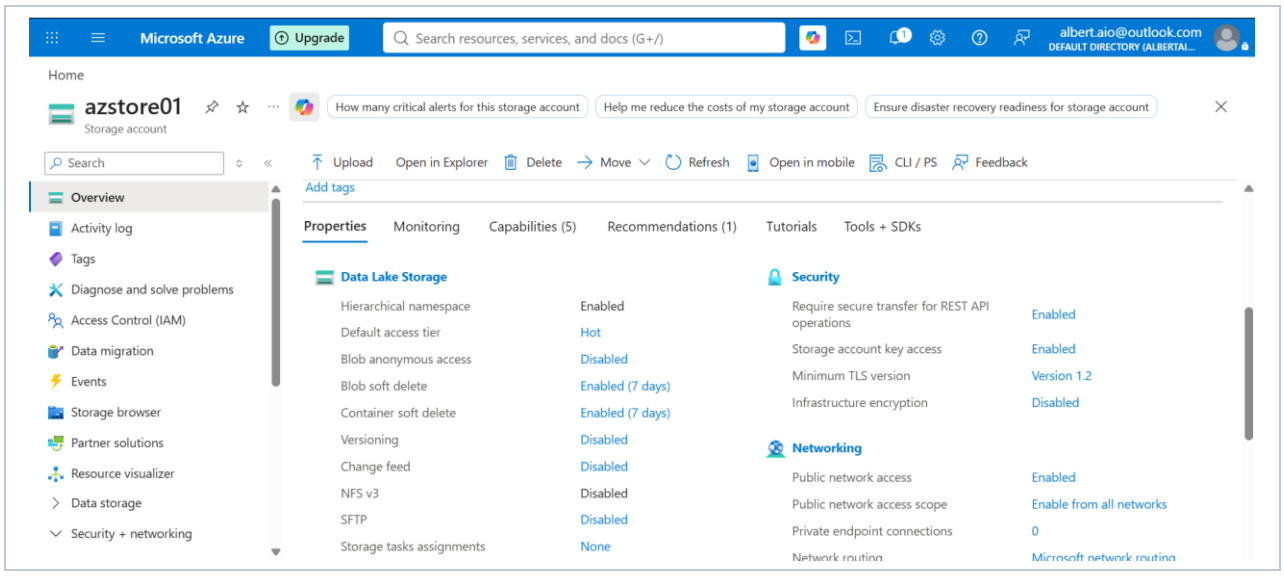


Figure 2: *Storage account azstore01: Data Lake Storage properties (hierarchical namespace enabled) and security settings.*

3.2 Containers

The account contains four containers: $logs (system), airlines, green-taxi and uber-taxi. The uber-taxi container, created on 14 October 2026, is the one used by this project.

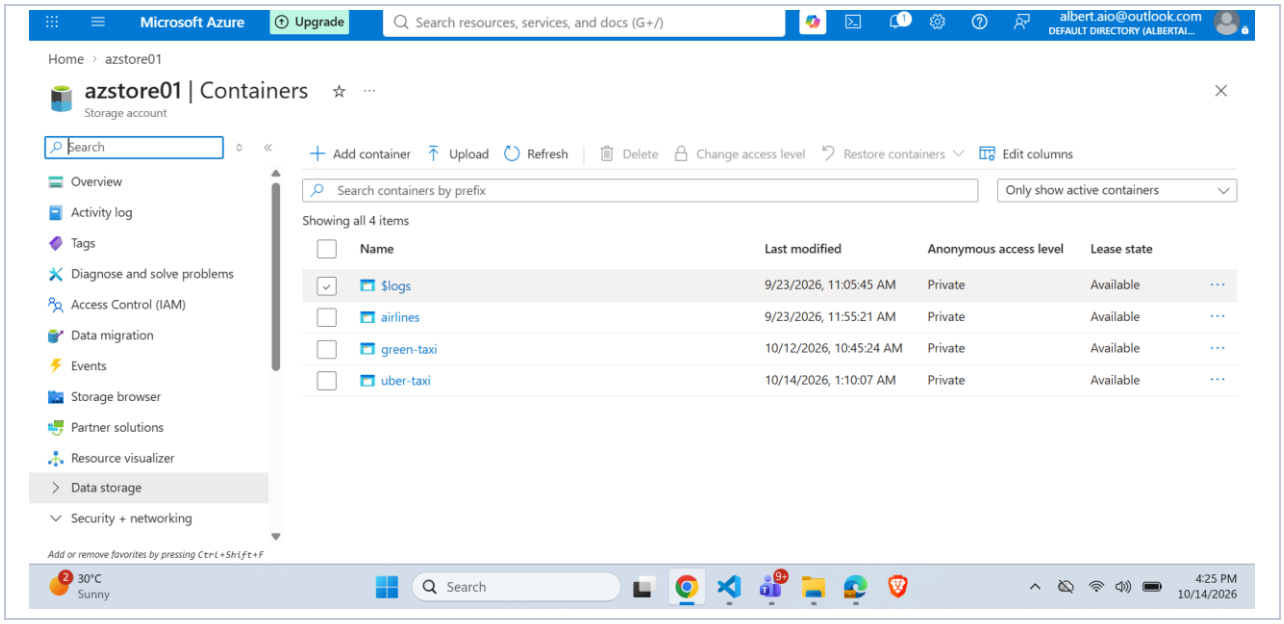


Figure 3: *Containers in azstore01; uber-taxi is the project container.*

3.3 Container contents

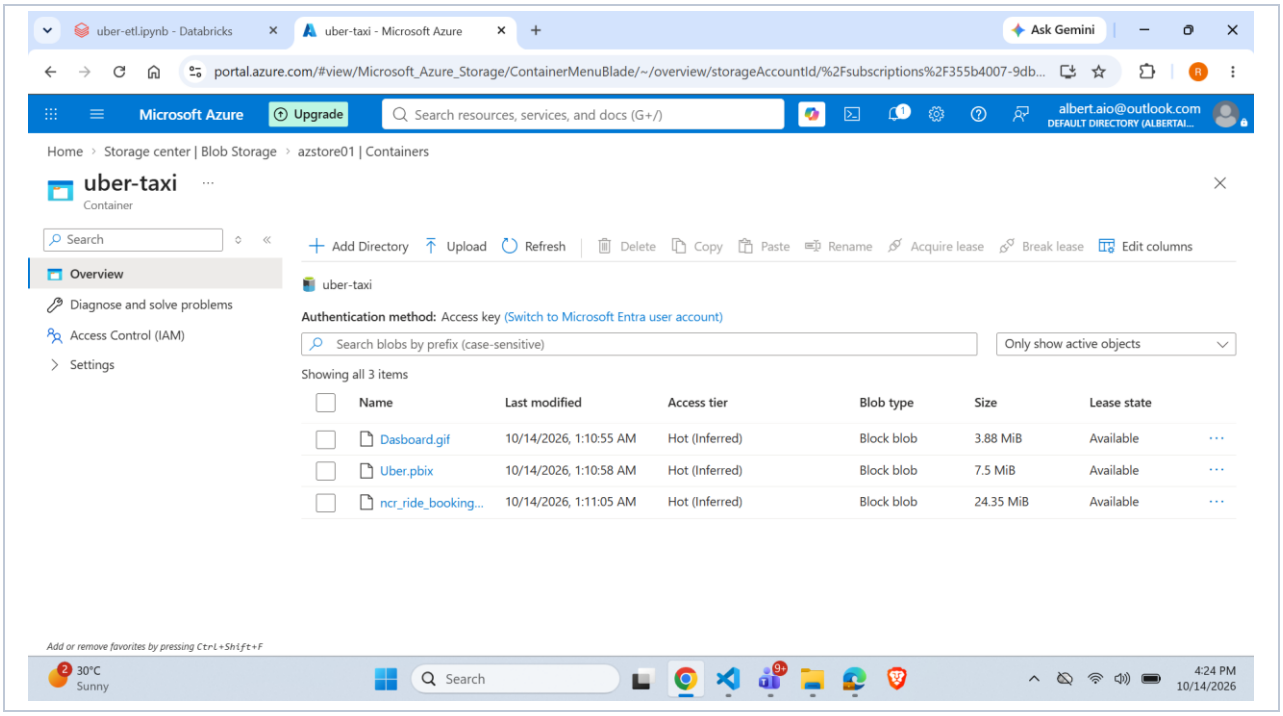


Figure 4: *Contents of the uber-taxi container: Dasboard.gif (3.88 MiB), Uber.pbix (7.5 MiB) and the source CSV ncr_ride_booking… (24.35 MiB).*

3.4 Storage hierarchy used by the code

All paths are built in src/config.py from the account name and container, so no path is hard-coded in the pipeline code.

Layer	Path (relative to abfss://uber-taxi@azstore01.dfs.core.windows.net)	Format
Source	/ncr_ride_bookings.csv	CSV with header
Bronze	/output/bronze	Delta
Silver	/output/silver	Delta
Gold – daily	/output/gold/daily_trip_summary	Delta
Gold – location	/output/gold/location_trip_summary	Delta
Gold – payment	/output/gold/payment_type_trip_summary	Delta

Note: The guideline suggests separate raw/, bronze/, silver/, gold/, quarantine/ and reports/ containers. This implementation uses a single container with an /output folder per layer, and has no quarantine or reports location yet (see Section 10).

4. Data Transformation Logic (Bronze, Silver, Gold)

4.1 Bronze – raw ingestion

- Reads the CSV with header=True and inferSchema=True after setting the storage account key on the Spark session.

- Replaces spaces in column names with underscores (for example Booking Value becomes Booking_Value).
- Adds an ingestion_timestamp column, coalesces to 2 partitions and writes a Delta table with overwrite and overwriteSchema.
- Re-reads the written Delta table and returns it, so downstream steps run on stored data.

4.2 Silver – cleaned and filtered

- Casts Booking_Value, Ride_Distance, Driver_Ratings and Customer_Rating to double (renamed booking_value, ride_distance, driver_rating, customer_rating).
- Removes duplicate records with dropDuplicates() and prints a per-column null count for inspection.
- Keeps only Completed bookings that have a non-null, positive booking value and ride distance.
- Derives pickup_date (from Date) and pickup_hour (hour of Time), then writes the Silver Delta table.

4.3 Gold – business aggregates

Three aggregate tables share the same metric set: total_trips, total_revenue, average_booking_value, average_ride_distance and average_driver_rating.

Gold table	Grouped by	Business question answered
daily_trip_summary	pickup_date	How do trips and revenue trend day by day?
location_trip_summary	Pickup_Location	Which pickup areas generate the most demand and revenue?
payment_type_trip_summary	Payment_Method	How is revenue split across UPI, cards and other methods?

4.4 Pipeline entry point

run_etl(spark) chains read_source → build_bronze → build_silver → build_gold and returns a status dictionary with row counts for every layer. This single function is what the etl agent calls.

```
run_etl(spark) -> {
  "status": "success",
  "bronze_rows": ..., "silver_rows": ...,
  "daily_summary_rows": ..., "location_summary_rows": ..., "payment_summary_rows": ...
}
```

5. Data Quality Framework

quality_checks.run_quality_checks(spark) loads the Silver Delta table and returns one structured dictionary. The pass/fail decision is deterministic and is made in code, not by the language model.

Check	Rule	Result field
Volume	Silver row count must be greater than zero	row_count
Completeness	Required columns contain no nulls	null_counts
Validity – duration	Duration not null and not negative	invalid_duration
Validity – distance	Distance not null and greater than zero	invalid_distance
Validity – amount	Amount not null and greater than zero	invalid_amount
Overall	passed only when every check above is clean	status = passed / failed

Note: In the submitted code the required-column list still uses taxi-trip names (pickup_datetime, dropoff_datetime, trip_distance, total_amount, passenger_count, trip_duration_minutes) carried over from an earlier green-taxi dataset. The Uber Silver table does not contain them, which is exactly why the quality agent raised UNRESOLVED_COLUMN in Figure 8. Mapping these checks to booking_value, ride_distance, Booking_ID and the status/flag rules is the first fix recommended in Section 10.

6. Databricks Environment and Test Results

6.1 Workspace layout

The project lives in the Databricks workspace myazdbworkspace under the user folder uber-taxi-agentic-etl, mirroring the required structure: data, logs, notebooks and src. The notebook uber-etl.ipynb orchestrates every test run.

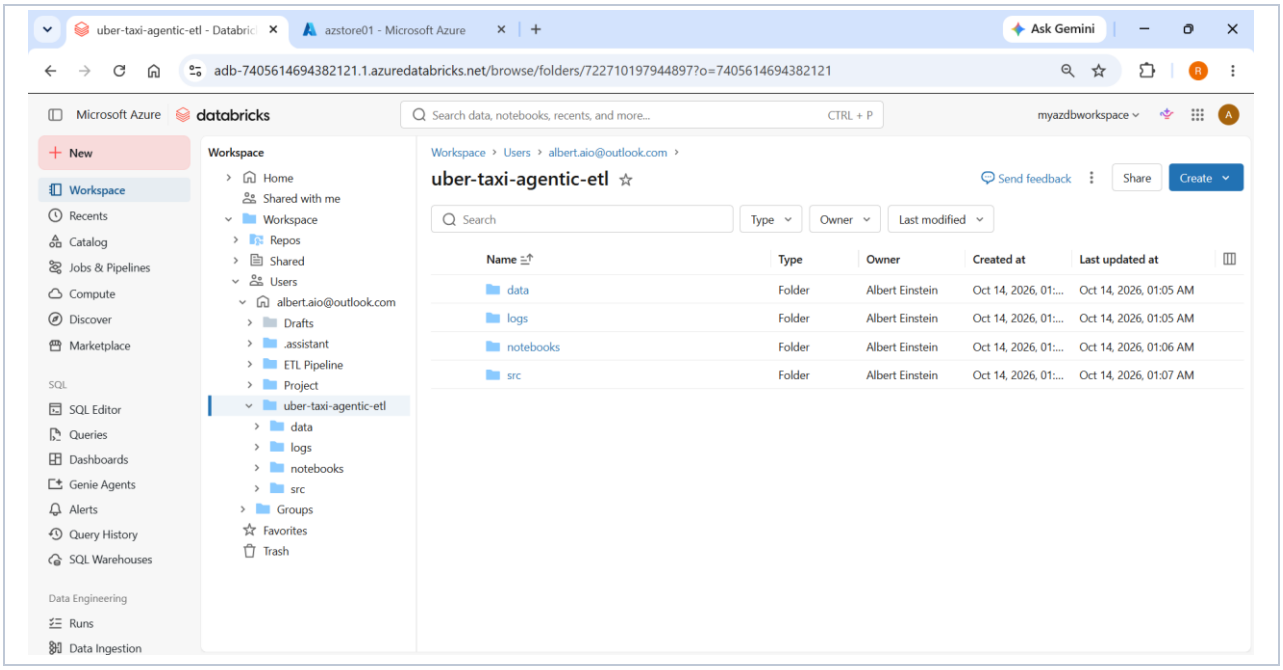


Figure 5: *Databricks workspace folder uber-taxi-agentic-etl containing data, logs, notebooks and src.*

6.2 Test notebook flow

Step	Action in uber-etl.ipynb
1	Install langchain, langgraph and langchain-openai at pinned versions, then restart Python
2	Add the src folder to sys.path and import run_etl
3	Run the full ETL and print the status dictionary
4	Preview Bronze, Silver and the three Gold tables with show()
5	Build the LangGraph agent graph and invoke it with natural-language queries
6	Render the graph with draw_mermaid_png()

6.3 Results observed

- **ETL execution.** The notebook ran run_etl and the supervisor graph; Spark executed 50 jobs in 56 seconds for the full-pipeline request (Figure 7).
- **Agent queries.** The multi-query test completed in 27 seconds and exposed the quality-check column mismatch (Figure 8).
- **Null counts.** The Silver null-count output in Figure 7 shows 0 for every column. This is expected for the current code: Spark reads the literal text “null” as an ordinary string, so isNull() does not count it. Converting the text to real nulls (nullValue='null' on read) is part of the planned fixes.

7. Agent Design: Roles, Tools, Workflow, Guardrails

7.1 Shared state

All nodes read and write a single typed state object, ETLState, with the fields: query, route, etl_result, quality_result, analytics_result and answer.

7.2 Agents

Node	Role	Tool / dependency
supervisor	Asks the LLM to return exactly one of run_etl, quality, analytics or explain. If the reply is not an allowed route, a keyword fallback chooses the route.	AzureChatOpenAI client
etl	Runs the full Bronze → Silver → Gold pipeline; catches exceptions and returns status failed with the error text.	run_etl(spark)
quality	Runs Silver validation; returns the structured result or status error.	run_quality_checks(spark)
analytics	Reads the Gold tables, computes overall totals and top locations, then asks the LLM to answer using only those computed metrics.	get_gold_metrics(spark)
explain	Answers conceptual questions; instructed not to claim it executed Spark jobs or inspected data.	AzureChatOpenAI client

7.3 Workflow and routing

The graph follows the guideline topology. A full run goes supervisor → etl → quality → analytics; validation only goes supervisor → quality; a question goes supervisor → explain. The etl node ends the run on failure, and the quality node continues to analytics only when the original route was run_etl.

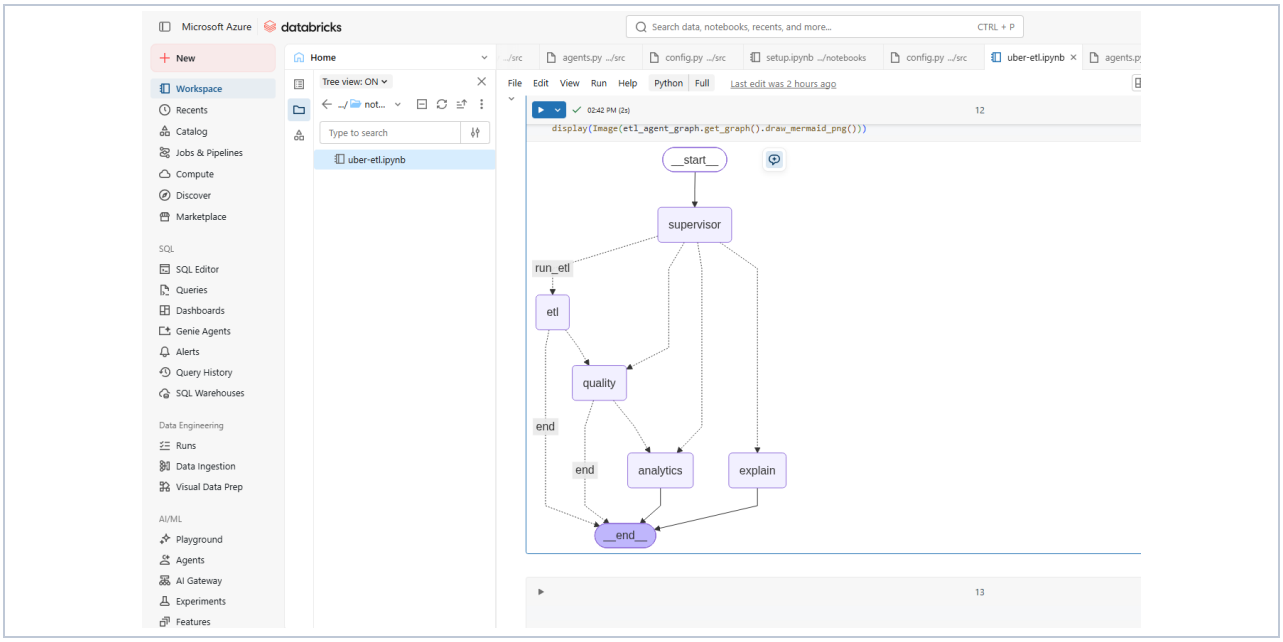


Figure 6: *LangGraph structure exported with get_graph().draw_mermaid_png(); dotted edges are conditional.*

7.4 Guardrails in the current implementation

- The supervisor output is constrained to an allow-list of four routes, with a deterministic keyword fallback.
- The analytics prompt forbids invented values and restricts the model to the computed Gold metrics.
- The explain prompt forbids claiming that jobs were run; every node wraps risky work in try/except.
- The client uses max_retries=2 and a 60-second timeout to bound cost and latency.

8. Foundry GPT Deployment and Prompt Design

Setting	Value
Platform	Microsoft Foundry project my-azureai-foundry
Model deployment	gpt-5-mini
Client	langchain_openai.AzureChatOpenAI (resource URL derived from the project endpoint)
API version	2024-12-01-preview
Temperature / retries / timeout	1 / 2 / 60 s
Authentication	API key read from config.py (Entra ID is the recommended upgrade)

8.1 Prompt design

- **Supervisor prompt:** role statement, an explicit closed set of options, one rule line per option, the user request, and “Return only the route name.” This keeps the output machine-parsable.
- **Analytics prompt:** role statement, the computed metrics as the only allowed evidence, “Do not invent values”, and a reminder that results reflect available data.
- **Explain prompt:** asks for a clear explanation and prohibits claims of having executed anything.

9. Fault Injection and Evaluation

The guideline asks for injected-fault testing and six evaluation metrics. The runs captured so far are functional tests of routing and execution; a formal fault-injection campaign has not yet been executed, so no detection or false-alarm figures are claimed here.

Evidence	What it shows	Reference
Full request routed to ETL	Supervisor chose run_etl; Spark ran 50 jobs in 56 s	Figure 7
Three-query test	Routes quality, analytics and explain were exercised in 27 s	Figure 8
Quality agent error	Mismatched column names surfaced as UNRESOLVED_COLUMN (pickup_datetime) and were reported instead of crashing the graph	Figure 8

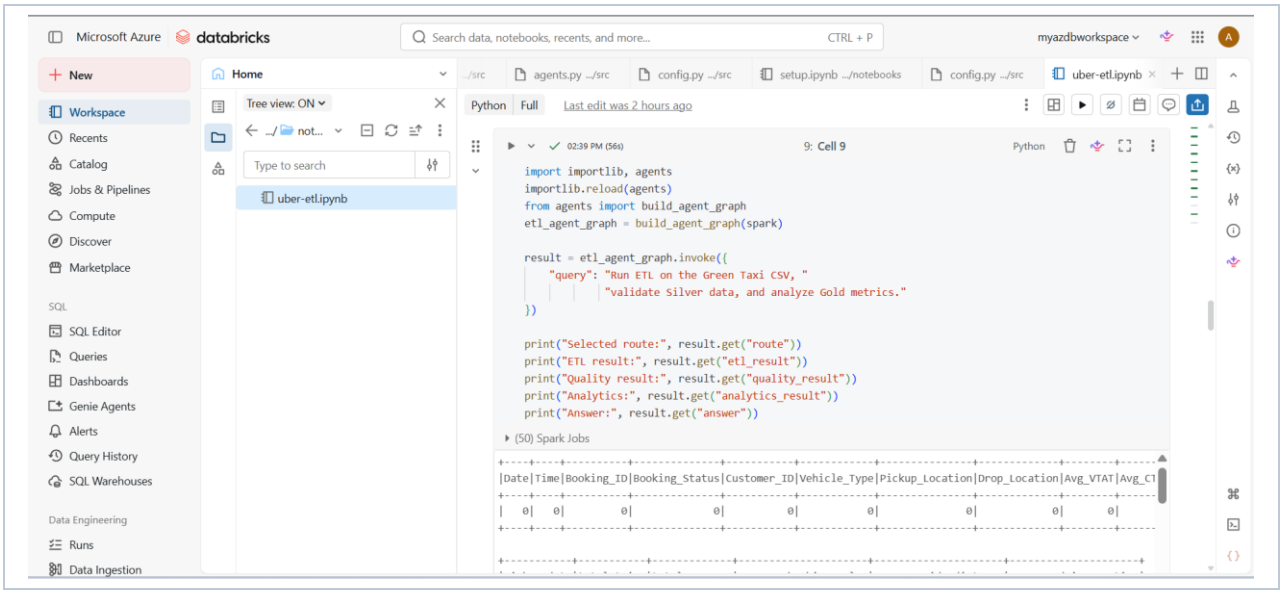


Figure 7: *Supervisor invocation: full-pipeline query, printed route/result fields and the Silver null-count table (50 Spark jobs, 56 s).*

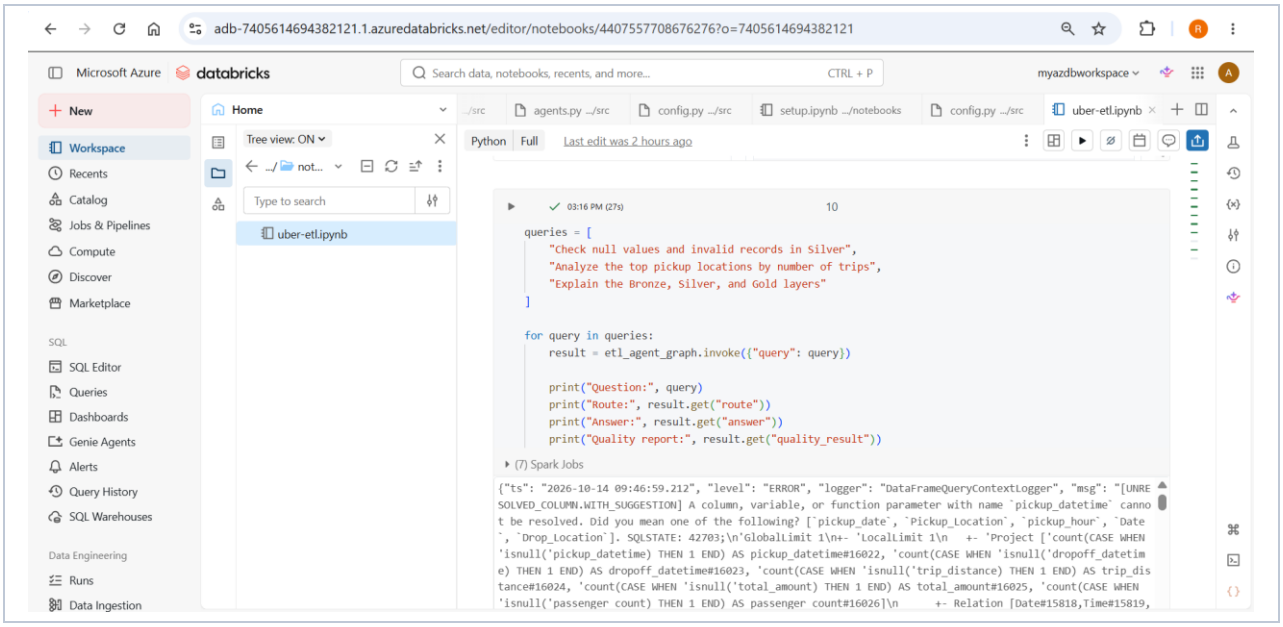


Figure 8: *Multi-query test (27 s). The quality agent returns an UNRESOLVED_COLUMN error because the check still references taxi-trip column names.*

9.1 Evaluation plan

Metric	How to measure	Status
Detection rate	Inject each fault (missing file, schema change, duplicates, null spike, negative value, rating > 5, flag mismatch, row drop, job failure) and count detections	Planned
False alarm rate	Run clean data repeatedly and count wrongly flagged runs	Planned
Recovery rate	Check the run stops at the right node with a correct explanation	Planned
Time saved	Compare manual validation time with agent-run time	Planned
Report quality	Manual review of explanation correctness and clarity	Planned
Cost and latency	Record tokens and seconds per run (latency already visible: 27–56 s)	Partly observed

10. Challenges, Limitations and Future Work

10.1 Gap analysis against the project guideline

Guideline requirement	Current implementation	Recommended next step
Explicit 21-column schema; text null → real null; strip quotes from IDs	inferSchema; text “null” stays a string; quotes not stripped	Define StructType; use nullValue='null'; regexp_replace on Booking ID and Customer ID
Bronze metadata: ingestion_timestamp, source_file, batch_id	Only ingestion_timestamp	Add input_file_name() and a batch_id; make loads idempotent and incremental by date
Silver keeps all statuses; invalid rows go to quarantine/	Keeps only Completed rows; others are dropped	Keep all statuses, derive is_cancelled, cancelled_by and cancellation_reason, write rejects to a quarantine Delta table
Silver timestamp and derived columns	pickup_date and pickup_hour only; Date not parsed	Parse dd-MM-yyyy and combine with Time into booking_timestamp; add day_of_week

Gold: cancellations, VTAT/CTAT, peak hours, ratings, incomplete rides	Daily, location and payment summaries	Add the remaining Gold tables; apply OPTIMIZE and Z-ordering
Quality: completeness, uniqueness, validity, consistency, timeliness, volume	Single Silver check with taxi-trip column names	Rewrite rules for Uber columns; return rule, layer, status, metric, threshold and sample failures
Gold reads in analytics	get_gold_metrics orders payment rows by payment_type, but the table column is Payment_Method	Change the orderBy column to Payment_Method
Databricks Job with retries and alerts	Notebook-driven runs	Package Bronze, Silver, Gold and quality as a Job; have the etl agent call the Jobs API
Quality → analytics only when hard rules pass	after_quality routes on the original route, not on the check status	Route on quality_result['status'] and add a retry limit
LLM severity classification with JSON output	LLM used for routing and answers	Send check results to GPT for critical/warning/info JSON; never let the model override a failed hard rule
Run report in reports/, run history, tool-call logging	Answers printed in the notebook	Persist state and write markdown/JSON reports to ADLS

10.2 Security

Action required: src/config.py currently contains the storage account key and the Foundry API key in plain text. These must be treated as exposed: rotate both keys now, remove them from the code and report, and load them from a Databricks secret scope or Azure Key Vault (or use Entra ID / a service principal). An .env.example listing variable names without values should replace them in the repository.

10.3 Other limitations

- Model risk: the supervisor depends on an LLM reply; the allow-list and keyword fallback reduce but do not remove wrong routing.
- Cost and latency: each agent call adds model tokens and 27–56 s of end-to-end time; token caps are not yet configured.
- Overwrite writes: every layer is rebuilt on each run, which is simple but not incremental.
- Small anomalies to clean up: the notebook query text mentions “Green Taxi CSV”, and one sys.path entry ends in “srcc”.

10.4 Future work

- Complete Phase 1 (schema, quarantine, full Silver rules, additional Gold tables, Databricks Job).
- Add the monitoring agent, human approval for destructive actions, and structured JSON decisions from the model.
- Run the fault-injection suite and publish the six evaluation metrics.

11. Conclusion

The project establishes a working foundation: an ADLS Gen2 account with the Uber dataset, a Databricks workspace organised to the required structure, a Delta-based Bronze, Silver and Gold pipeline, and a LangGraph multi-agent layer on a Microsoft Foundry model that routes requests, runs the pipeline and

answers analytical and conceptual questions. The screenshots confirm each stage running. The gap analysis in Section 10 gives a short, concrete list of changes that will bring the implementation fully in line with the guideline and make the evaluation results measurable.

Appendix: Source Code Map

File	Responsibility
src/config.py	Storage account, container, abfss paths per layer, Foundry endpoint and model name
src/etl_pipeline.py	read_source, build_bronze, build_silver, build_gold, run_etl
src/quality_checks.py	run_quality_checks – Silver validation returning a structured dictionary
src/gold_analytics.py	get_gold_metrics – overall totals, top 10 locations, payment summary
src/agents.py	AzureChatOpenAI client, ETLState, supervisor/etl/quality/analytics/explain nodes, build_agent_graph
notebooks/uber-etl.ipynb	Install dependencies, run ETL, preview tables, invoke agents, plot the graph